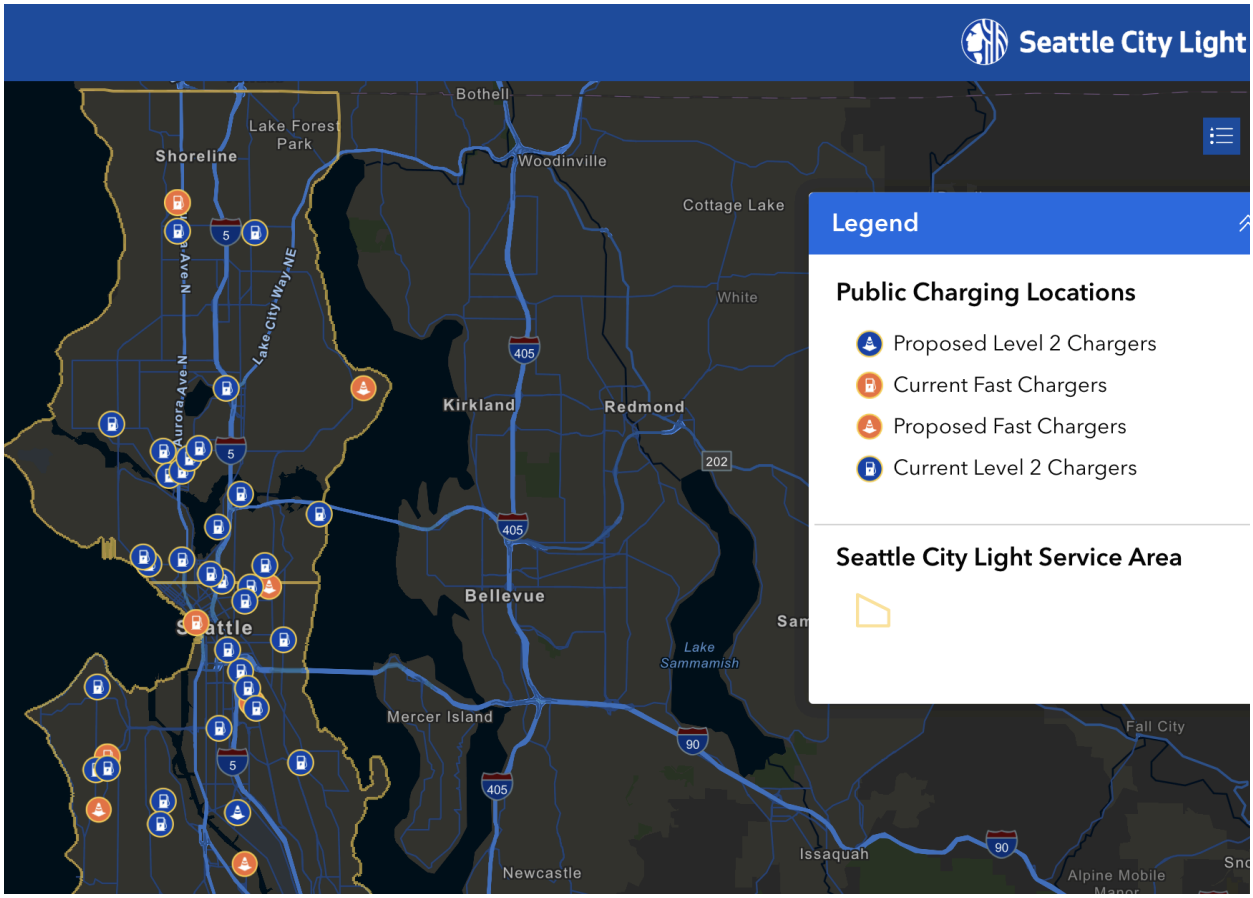
UIUC GGIS 407 - Summer 2026 - Ryan Rickerts

Electric Vehicle Charger Site Selection

WASHINGTON STATE SPATIAL OPTIMIZATION



Introduction

To increase adoption of electric vehicles (EVs) and reduce society's usage of fossil fuels for solving our global warming problem, we must increase the number of publicly available charging stations. This reduces "range anxiety" which prevents people from committing to fully electric vehicles.

My project presumes a government entity, Washington State, has the budget to place 14-16 additional EV chargers on public roads. As their spatial data analysis consultant, I have been tasked with using my HPC capabilities & programming skills to help them select the optimum locations.

Problem Statement

My hypothetical funding source, the state government, has contracted with my employer to help them find “charging deserts” where people in their EVs have been historically stranded out of battery charge, indicating a demand node or “charging desert” that could be addressed using the budget for new charging infrastructure.

As I have previously solved this problem for just the Puget Sound Area, now the problem has become how to distribute the state resources more fairly, i.e. not locating all chargers in the high population metro area of Seattle/Tacoma/Everett (aka Puget Sound Region) to help appear unbiased and more democratic as a government entity. This round of the data analytics exercise involves placing 2 chargers in each of 8 state’s sub-regions based on a demand signal found within that bounding box in the AAA data.

Posing this as a P-center problem, where distance must be minimized along the road network, we can find the optimum locations to meet the unweighted demand (i.e. minimize distance for each incident).

For reporting to my superiors, I have prepared a series of plots overlaying existing charging infrastructure, major roads in the network, and AAA-reported outage locations in Washington State and the proposed new locations for up to 16 charging stations. These images have been included in this report, also.

Data Sources & Pre-processing Steps

The following data sources were utilized to enable this spatial analysis using Jupyter notebook on the CyberGISX platform.

AAA Out of Charge Incidents

This interest of mine goes back to 2023 when I thought about using the Google Maps API and some yet unknown ML technique to determine the optimum charger locations, perhaps train a model that could be utilized for dynamic outputs as more funding for EV chargers emerged from federal or state initiatives.

At that time, AAA gave an online presentation I attended talking about using their dataset for service calls nationally in the U.S. to solve the same problem. This slide is from their sales pitch. I contacted their company representative and purchased the data for Washington State for 1 year (Nov. ‘22 to Nov. ‘23) and this cost me $200.

At that price for an unsponsored academic project, I have decided to re-use the .csv datafile they sent me. It contains the latitude and longitude of the vehicle pickup, make/model of the vehicle, the cause for the service call (e.g. flat tire), etc. I had to filter this data for only out-of-charge incidents which reduced the number of interesting points in the Seattle Metro area to 39 events.

This is perhaps statistically insignificant but adequate for a proof-of-concept approach. With a greater budget, a larger, multi-year dataset could be purchased from AAA for better pattern analysis.

Washington Road Network

The road network for the entirety of Washington State was downloaded for free from the Geofabrik GmbH website which extracts and compiles OpenStreetMap project data. You can drill into sub-regions by country and further into the sub-region of interest. Washington is the smallest resolution offered and overlaps perfectly with my AAA dataset. However, I had difficulty loading this entire .osm.pbf file into the kernel memory in the CyberGISX or I-GUIDE environment, a limitation I will discuss in the Pre-processing section below.

Existing Charger Infrastructure

The last and perhaps most critical data source came from the U.S. Department of Energy Alternative Fuels Data Center Data Downloads page. This, like the road network data, is filterable by state of interest with public & functioning infrastructure only. It also allows for a historical query, so I selected Dec. 31, 2023 to match the AAA dataset. The data format is provided in a GeoJSON with lat/lon values for the stations.

Offline Pre-processing Step

I was aware of constraints with my project plan due to memory limits in the CyberGISX environment itself. Multiple attempts and approaches in the past to getting the Washington road network into OSM had failed and caused kernel restarts.

Therefore, I installed the Osmosis library locally using Homebrew, a command line interface (CLI) for processing OSM data (sponsored by the OpenStreetMap project itself), in the terminal. This will allow me to solve an interesting set of questions with `pyrosm`, `networkx`, `pulp`, and other python libraries I have learned about in a previous course, GGIS 480.

The filtering by tags command is:

```
osmium tags-filter washington-260425.osm.pbf \
w/highway=motorway,motorway_link,trunk,trunk_link,primary,primary_link,secondar
y,secondary_link,tertiary,tertiary_link \
-o washington-drivable.osm.pbf
```

This reduces the road network to primary, secondary, and tertiary motorways and their links which is critical for solving routing problems on the network accurately, but produced only a 13MB file which is easier to upload into the CyberGISX environment and lighter weight for processing, as well.

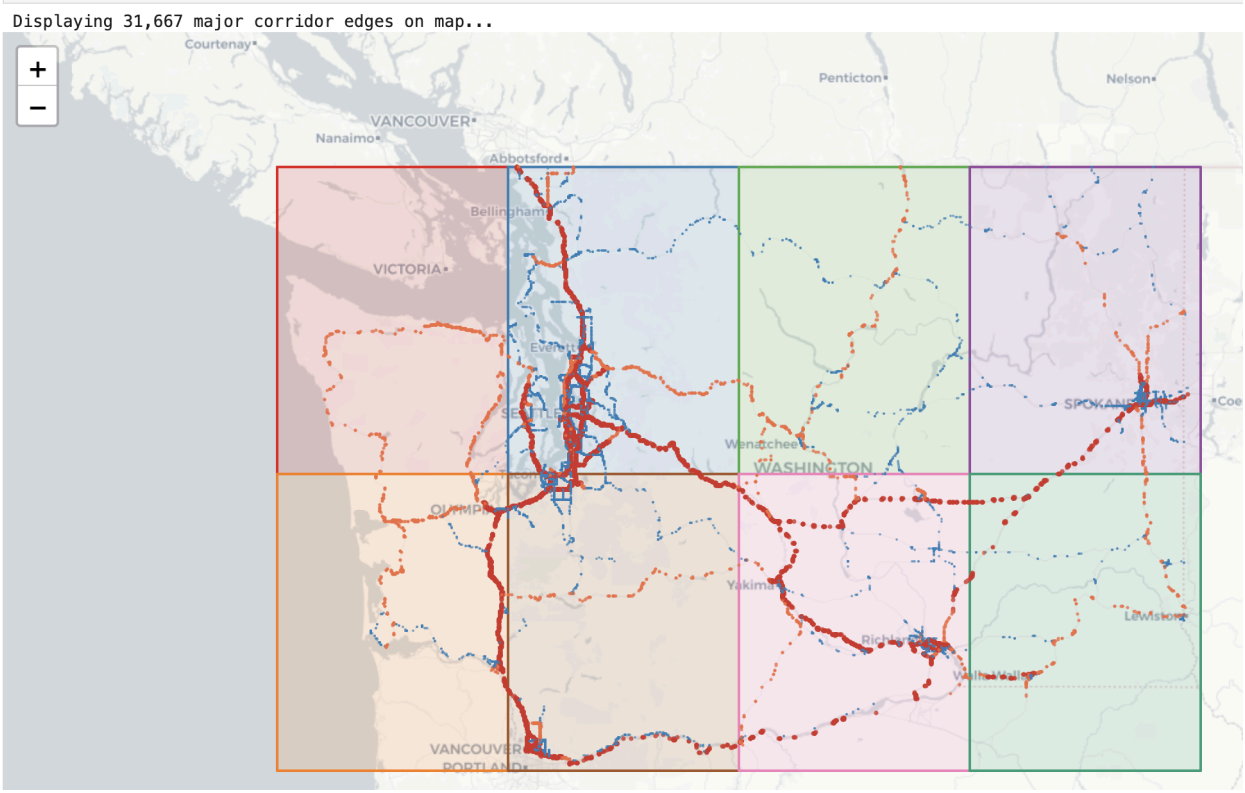
Divide & Conquer Step

To demonstrate the potential to run this algorithm in a high-performance compute environment, we must decide how to carve up the problem. Since the task was already defined as delivering up to 2 new charging stations to each of 8 regions of the state, we can use this 2x4 grid to process the data separately in the HPC domain. That grid is as follows:

```
NW Olympic Peninsula: Port Angeles, Strait of Juan de Fuca
Puget Sound North: Seattle, Everett, Bellingham, N. Cascades
North Central: Wenatchee, Chelan, Okanogan
NE Spokane: Spokane, Colville, ID border north
SW Pacific Coast: Grays Harbor, Longview, SW Coast
```

```
South Cascades: Tacoma south, Olympia, Mt Rainier, Vancouver
SE Tri Cities: Yakima, Tri-Cities, Columbia Basin
SE Palouse: Pullman, Walla Walla, ID border south
```

When rendered on a Folium map (with a sample of the road network overlaid), it looks like this, where we can clearly see the road density in the Puget Sound area is the highest:



The third step in the pre-processing targets a parallel compute paradigm on HPC, which I do not yet have access to reach the real hardware, so that work must be a follow-up project. Instead, I prepared my approach such that the calculations could be distributed, but in the virtual environment provided for our coursework, parallel processing still causes memory exhaustion crashes in the optimization step.

This step, with a ProcessPoolExecutor, succeeded in processing the WA drivable PBF (Protocolbuffer Binary Format) file generated above with Osmosis by bounding box or `bbox` using a method called process_region().

```
# Process all 8 bounding boxes simultaneously
with ProcessPoolExecutor(max_workers=MAX_WORKERS) as executor:
    results = list(executor.map(process_region, wa_regions_8.items()))
```

This method must be available in a .py file on disk since the child processes do not have the memory loaded from the running notebook or parent process with this method defined.

This region processor or worker method does one simple but very important thing: load the PBF file by one of the 8 pre-defined bounding boxes, execute this code on it, and save out a Parquet file:

```
osm = OSM(PBF_PATH, bounding_box=geom_bbox)
nodes, edges = osm.get_network(network_type="driving", nodes=True)
```

This should allow for later execution of the optimization problem also by parallel computation on an extracted set of nodes and edges for a driving network in that region. The files are saved in a sub-directory called `pq_data`. Also, in the case of a kernel crash, this is an intermediate state of our network processing saved on disk.

This approach can be viewed in the map above with edges obtained from the Parquet files as:

```
statewide_edges = pd.concat([gpd.read_parquet(f) for f in edge_files])
       .drop_duplicates(subset=["id"])
```

When the full state is read back into memory, we get:

```
Loading and deduplicating regional Parquet files...
Total Unique Nodes: 940,465
Total Unique Edges: 92,666
```

Optimization Step: the P-Median Problem

The next step is to load three things into memory:

- a statewide NetworkX MultiDiGraph from the nodes and edges read in from the Parquet files
- all AAA Out of Charge incident locations
- existing EV charger infrastructure from the .geojson file

and snap both of those data points to the graph. This prevents failed routing problems I experienced in my prior attempt. We can only run minimum distance calculations from origin to potential destination on the network.

This is where the real work happens, but I was unable to parallel process these in this constrained resource environment. It crashed in the Puget Sound Region calculation every time. The external .py file with a method called:

```
def optimize_region_pulp(args):
    region_name, core_bbox, incidents_gdf, chargers_gdf,...
```

where the parameters needed for each sub-region are passed in as args.

The job in the main notebook is still defined in a way the LProblem definition and solver could be a significantly parallel portion and thereby raising our computational efficiency. The external file still reads the Parquet files for each sub-region from disk, but for now the code is executed serially:

```
# number of EV charger sites to be distributed evenly in 8 WA regions
DEFAULT_QUOTA_PER_REGION = 2

# this is ready for parallel execution once we have access
worker_tasks = [
    (region_name, bbox, clean_incidents, clean_chargers,
DEFAULT_QUOTA_PER_REGION)
    for region_name, bbox in wa_regions_8.items()
]
```

```
for task in worker_tasks:
    region_name = task[0]
    res = optimize_region_pulp(task)
```

and the result after about 10 minutes:

```
OPTIMIZATION COMPLETE IN 614.56 SECONDS
nw_olympic_peninsula   | Selected Sites: 2 | Node IDs: [5494450248,47189579]
puget_sound_north      | Selected Sites: 2 | Node IDs: [2927137481,1315578478]
north_central          | Selected Sites: 2 | Node IDs: [50051615,47066567]
ne_spokane             | Selected Sites: 2 | Node IDs: [47973570,47895913]
sw_pacific_coast       | Selected Sites: 2 | Node IDs: [51408922,49928916]
south_cascades         | Selected Sites: 2 | Node IDs: [110397230,369930272]
se_tri_cities          | Selected Sites: 2 | Node IDs: [47965994,48189294]
se_palouse             | Selected Sites: 0 | Node IDs: []

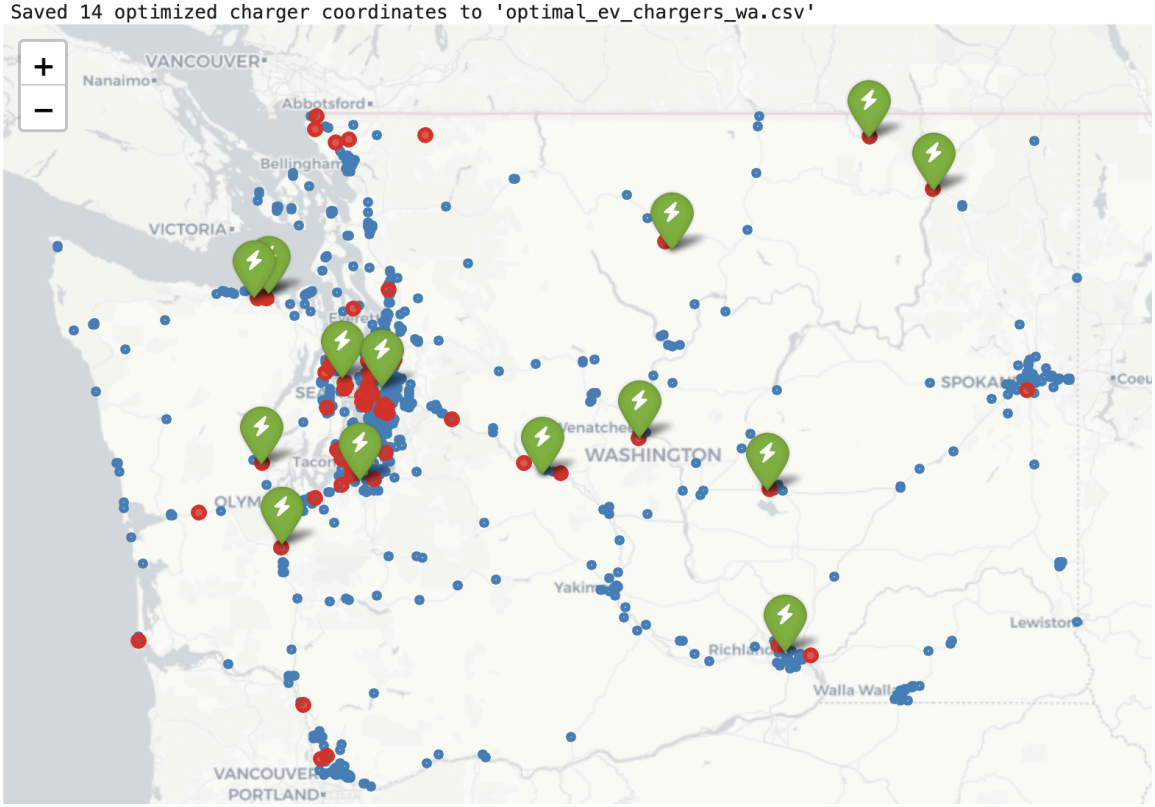
Total Optimal Charger Sites Selected Statewide: 14
```

with the surprise result of no chargers in the SE Palouse because there were no outage incidents in the sub-region, i.e. no demand nodes. *

Displaying the Results

The last part is the most fun, the map showing chargers in blue, out of charge incidents in red, and the proposed new sites with green lightning bubbles. This map can be zoomed in on the notebook to check for accuracy or a “gut check” i.e good distribution away from existing chargers (minimum distance set in code was 4km) and good coverage near the outage incidents. We also specified they must be a node with at least 3 edges attached, meaning an intersection, thereby servicing more people in that area of demand.

```
Total Optimal Charger Sites Selected Statewide: 14
Saved 14 optimized charger coordinates to 'optimal_ev_chargers_wa.csv'
```



In some cases, like the NE Spokane area, the chargers were placed exactly where the outage occurred. This is technically correct, but in very remote areas with sparse coverage, the proper site would likely be hand selected based on available electrical infrastructure.

Conclusion

The problem tackled here is somewhat unrelated to the coursework we covered in GGIS 407, but it is still appropriate within the domain of CyberGISX and spatial data analysis and stitches several courses together: GGIS 480 and this one.

In order to tackle a charging location proximity problem, I effectively applied the P-Median objective function to a statewide dataset. It minimizes the total (or average) network travel distance across all out-of-charge incidents with the goal of overall system efficiency and cumulative miles saved for all stranded drivers. This means, with these chargers in place, the drivers could have potentially reached a station before running out of charge and not required a AAA tow driver to rescue them.

With a long-standing interest in this problem area, I was aspirational to find the parallel processing tools right at my fingertips after taking this course. Unfortunately, the virtualization layer provided to students through the CyberGISX and I-GUIDE are still inadequate for this large (or even larger) dataset (aka. computational intensity) combined with a high computational complexity of the optimization problem.

The lack of chargers sited in the SE Palouse area demonstrates my result is not contrived, although you could argue no one drives through that area with an EV if it has a reputation as a charging desert, making this a chicken vs. egg problem. Which comes first? Chargers or Out of Charge Incidents?

With a project sponsorship, the dataset and accuracy could be increased to provide deeper proof of the ideal locations in Washington State. With access to the HPC resources at UIUC, we could also expand the problem to be solved on a national scale with quicker iteration.

The seeds of a great idea that were planted in the previous semester at UIUC continue to grow here and have a more refined & powerful approach. I hope to continue on this project in the coming weeks independently with the help of Dr. Anand Padmanabhan and the GGIS department.

* **ACADEMIC INTEGRITY DISCLOSURE**: Google Gemini was used to help troubleshoot and generate some portion of this code, although all concepts and execution are understood by me.